Experiment No. 8:
Arrays in C – Part II

Objectives

The objectives of this lab are:

1. To understand searching techniques used in arrays.
2. To implement linear search and binary search using C programming.
3. To learn the declaration, initialization and manipulation of 2-D arrays in C.
4. To understand how 2-D arrays are stored in memory.
5. To implement 2-D arrays in solving computational problems.
6. To understand transpose, traversal and searching operations using 2-D arrays.
7. To develop C programs using nested loops for handling 2-D data.

Prerequisites

Following are the prerequisites of this lab are:

1. Understanding of data types in C programming language.
2. Basic knowledge of variables, loops, control statements and functions in C.
3. Understanding of 1-D arrays in C.
4. Basic knowledge of memory addressing and sorting of arrays.
5. Read section 6.10 and section 6.11 from textbook (C-How to Program, 9 Ed.)

Introduction

Searching Techniques in Arrays
The process of finding the key value inside an array is called searching.
Two commonly used techniques are:
1. Linear Search Technique
2. Binary Search Technique

Linear Search Technique
Linear search is the most common and easy technique to search a key value in an array. In this technique, each element of the array is checked until the desired value is found or the entire array has been traversed. Because the array is not sorted, the search has no information about where the key might be located. This means the key could appear anywhere in the array (at the beginning, middle, or end) with equal probability. If the key value is found, C program returns the element's subscript; otherwise, it returns -1 (an invalid subscript) for unsuccessful search. For understanding observe the output of the following code.

Example:

```
#include <stdio.h>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int key = 30;  // Change the value of key to 55 and observe the output
    int index = -1;

    for(int i = 0; i < 5; i++) {
        if(arr[i] == key) {
            index = i;   // Save the index
            break;
        }
    }

    printf("Index: %d\n", index);
    return index;   // return the index
}
```

Output:

```
Index: 2
```

Note: Linear search examines elements one by one; therefore, it is a slow approach for large arrays. Even if the array is sorted, linear search does not take advantage of ordering. Because it just compares the target value with the current element.

Binary Search Technique

Binary search is more efficient but complex searching technique. It works by repeatedly dividing the search interval into two halves and comparing the key value with the middle element of the current interval. If the key value matches the middle element, the search is complete. If the key value is smaller, the search continues in the left half; if larger, it proceeds in the right half. This searching sequence is followed if the array is sorted in ascending order, in the case of descending order adjust the comparison logic. This halving process continues until the key is found or the search space becomes empty. For better understanding observe the following code.

Example:

```
#include <stdio.h>
int main() {
    // Declare and initialize a sorted array of 7 elements
    int arr[7] = {5, 10, 15, 20, 25, 30, 35};
    int Left = 0, Right = 6;        // Initialize the search range (indices)
    int mid;                       // To store middle index
```

```
    int key = 25;
    int i;
    int found = 0;

    // Print subscripts
    printf(" Subscripts:\n");
    for(i = 0; i < 7; i++)
        printf("%3d ", i);
    printf("\n");

    // Print full array elements
    printf(" Elements:\n");
    for(i = 0; i < 7; i++)
        printf("%3d ", arr[i]);
    printf("\n");

    // Binary search with fully visual steps
    while(left <= right) {
        mid = (left + right) / 2;

        for(i = 0; i < 7; i++) {
            if(i < left || i > right)
                printf("    ");              // blank for elements outside range
            else if(i == mid)
                printf("*%2d*", arr[i]);     // highlight middle
            else
                printf("%3d ", arr[i]);      // show elements in current range
        }
        printf("\n");

        if(arr[mid] == key) {
            found = 1;
            break;
        }
        else if(key < arr[mid])
            right = mid - 1;
        else
            left = mid + 1;
    }
    if(found)
        printf("\n%d found at subscript %d\n", key, mid);
```

```
    else
        printf("\n%d not found in the array\n", key);
    return 0;
}
```

Output

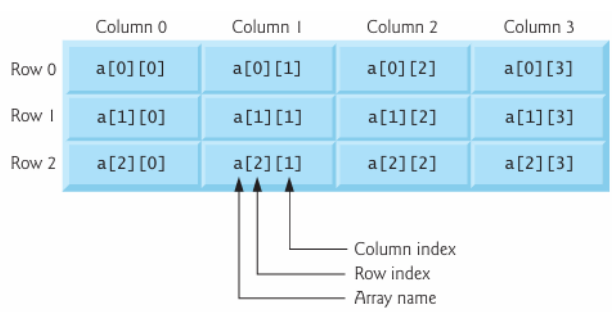
```
Subscripts:
 0  1  2  3  4  5  6
 Elements:
 5  10  15  20  25  30  35
 5  10  15 *20* 25  30  35
            25 *30* 35
           *25*

25 found at subscript 4
```

Note: Binary search requires the array to be sorted because it makes decisions based on the order of elements. If the array is not sorted, the algorithm cannot correctly determine which half of the array to search next, and the search may fail.

Multidimensional Arrays (2-D Arrays)

Arrays can have multiple subscripts. Arrays that require two subscripts to identify a particular element commonly are called two-dimensional (2-D) arrays. Multidimensional arrays can have more than two subscripts. However, in this lab, the focus will be on the implementation of 2-D arrays in the C language.

The 2D array is organized as matrices which can be represented as the collection of rows and columns. The following diagram illustrates a two-dimensional array:



Defining & Initialization of 2-D Array

A 2-D array is defined by declaring data its type, name, and size (number of rows and columns). Initialization of an array means assigning value to its element. A 2-D array can be initialized at the time of declaration or later by assigning values to individual elements using their index positions.

Such as;

DataType arrayName[Rows][Columns] ➡ int matrix[3][5];

Syntax for initialization of an Array

```
//Deceleration of an Array
DataType arrayName[rows][columns];

//Deceleration &Initialization of an Array at Same Time
//Row-wise Initialization
DataType arrayName[rows][columns]= {
    {R1C1, R1C2, R1C3, R1C4, ..., R1Cn},    // row 0
    {R2C1, R2C2, R2C3, R2C4, ..., R2Cn},   // row 1
    {R3C1, R3C2, R3C3, R3C4, ..., R3Cn},  // row 2
    ...
    {RmC1, RmC2, RmC3, RmC4, ..., RmCn}  // mth row
};

// Initialization of an Array Using Index Position
DataType arrayName[rows][columns];
arrayName[row_index][column_index] = value;
...
```

Note: Initialization of 2-D can be done in a continuous manner by assign all values with-in nested braces, and can also initialize partially such that the remaining elements has 0 value.

Example:

```
#include <stdio.h>

int main() {
    int num[2][3] = {{1, 2, 3}, {4, 5, 6}};

    printf("2-D Array Elements:\n");

    for(int r = 0; r < 2; r++) {
        for(int c = 0; c < 3; c++) {
            printf("%d ", num[r][c]);
        }
        printf("\n");
    }

    return 0;
}
```

Output:

```
2-D Array Elements:
1 2 3
4 5 6
```

Using a Loop to Set the Array's Element

Since a 2-D array contains multiple elements arranged in rows and columns, each element is accessed using two indices: one for the row and one for the column. To access all elements in sequence, use nested loops in which an outer loop to traverse the rows and an inner loop to traverse the columns. The outer loop runs from index 0 to rows – 1, and for each row, the inner loop runs from index 0 to columns – 1, allowing us to access and manipulate every element in the array.

Example

```c
#include <stdio.h>
int main() {
    int rows = 3, cols = 4;
    int arr[3][4];
    // Using nested loops to set values
    for(int i = 0; i < rows; i++) {       // loop over rows
        for(int j = 0; j < cols; j++) {   // loop over columns
            arr[i][j] = (i+1) * (j+1);   //  Elemnet= row_index * column_index
        }
    }
    // Print array to verify
    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++) {
            printf("%d\t", arr[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Output:

```
1    2    3    4
2    4    6    8
3    6    9    12
```

Element-level Manipulation on 2-D Arrays

Manipulation operations on 2-D arrays allow us to modify the contents of an array by adding, removing, or updating elements. The three common operations are: Insertion, Deletion, and Replacement.

Example:

```
#include <stdio.h>
int main() {

    int arr[10][10] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9,10,11,12}
    };
    int rows = 3, cols = 4;
    int newRowIndex = 1;
    int newRow[] = {100, 200, 300, 400};

    // shift rows down
    for(int i = rows; i > newRowIndex; i--) {
        for(int j = 0; j < cols; j++) {
            arr[i][j] = arr[i-1][j];
        }
    }
    // insert new row
    for(int j = 0; j < cols; j++) {
        arr[newRowIndex][j] = newRow[j];
    }
    rows++;

    // print result
    printf("After Inserting New Row:\n");
    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < cols; j++)
             printf("%4d ", arr[i][j]);
        printf("\n");
    }

    return 0;
}
```

Output:

```
After Inserting New Row:
  1   2   3   4
100 200 300 400
  5   6   7   8
  9  10  11  12
```

Example:

```c
#include <stdio.h>

int main() {
    int arr[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };
    // Replace the element of the array
    int r = 1, c = 1; // middle element
    arr[r][c] = 500;  // replace

    printf("After Updating Element at (1,1):\n");

    for(int i = 0; i < 3; i++) {
        for(int j = 0; j < 3; j++)
            printf("%4d ", arr[i][j]);
        printf("\n");
    }
    return 0;
}
```

Output:

```
After Updating Element at (1,1):
  1   2   3
  4 500   6
  7   8   9
```

Passing a 2-D Array to a Function

Passing a two-dimensional (2-D) array to a function works in a similar way as passing a 1-D array, because arrays in C are always passed by reference. This means that instead of copying all the elements of the array,

only the memory address of the first element is sent to the function. Since the function receives the actual memory location of the 2-D array, any changes made inside the function directly affect the original array. In a 2-D array, elements are stored in row-major order, so the array name represents the base address of the first row.

When passing a 2-D array, C requires that the number of columns must always be specified, because the compiler needs this information to correctly compute the memory address of each element using indexing formulas. However, the number of rows can be passed separately as another argument. Similar to 1-D arrays, a 2-D array can be passed to a function in two ways: using standard array notation (e.g., arr[][3]) or using pointer notation (e.g., int (*arr)[3]). Pointer-based methods will be discussed in detail in later manuals when the concept of pointers is fully introduced. For now, it is important to understand that passing a 2-D array simply means providing access to its real memory location, allowing the function to read or modify the original matrix directly.

Example:

```c
#include <stdio.h>
// Function to print matrix
void printMatrix(int A[][3], int rows) {
    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < 3; j++)
            printf("%d ", A[i][j]);
        printf("\n");
    }
}
// Function to compute sum of all elements
int sumMatrix(int A[][3], int rows) {
    int sum = 0;
    for(int i = 0; i < rows; i++) {
        for(int j = 0; j < 3; j++)
            sum += A[i][j];
    }
    return sum;
}
int main() {
    int M[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };
    printf("Matrix:\n");
    printMatrix(M, 2);
    int total = sumMatrix(M, 2);
    printf("\nSum of all elements = %d\n", total);
    return 0;
}
```

Output:

```
Matrix:
1 2 3
4 5 6

Sum of all elements = 21
```

Lab Tasks

Task 1

A class stores student roll numbers in ascending order along with their marks. Write a C program to search for a roll number using binary search and print the corresponding marks if found or not.

C Code

task1.c
~/Desktop/PF_Labs/Lab08_2025_EE_016

```c
#include <stdio.h>

int main() {
    int rollNumbers[] = {101, 105, 112, 118, 120, 125, 130};
    int marks[] = {85, 92, 76, 88, 95, 81, 90};
    int n = sizeof(rollNumbers) / sizeof(rollNumbers[0]);
    int key, mid;
    int left = 0;
    int right = n - 1;
    int found = 0;

    printf("Enter roll number to search: ");
    scanf("%d", &key);

    while (left <= right) {
        mid = (left + right) / 2;

        if (rollNumbers[mid] == key) {
            found = 1;
            break;
        }
        else if (key < rollNumbers[mid]) {
            right = mid - 1;
        }
        else {
            left = mid + 1;
        }
    }
```

```
    if (found) {
        printf("Roll Number %d found! Marks: %d\n", key, marks[mid]);
    } else {
        printf("Roll Number %d not found in the record.\n", key);
    }

    return 0;
}
```

Output

```
muhammad-umar@umarrsthinkpad:~/Desktop/PF_Labs/Lab08_2025_EE_016$ gcc task1.c -o task1
muhammad-umar@umarrsthinkpad:~/Desktop/PF_Labs/Lab08_2025_EE_016$ ./task1
Enter roll number to search: 188
Roll Number 188 not found in the record.
muhammad-umar@umarrsthinkpad:~/Desktop/PF_Labs/Lab08_2025_EE_016$ ./task1
Enter roll number to search: 118
Roll Number 118 found! Marks: 88
muhammad-umar@umarrsthinkpad:~/Desktop/PF_Labs/Lab08_2025_EE_016$
```

Task 2

Write a C program to multiply two matrices of size 3×3.

C Code

Open • task2.c ~/Desktop/PF_Labs/Lab08_2025_EE_016

```
#include <stdio.h>

int main() {
    int A[3][3] = { {1, 2, 3},
                    {4, 5, 6},
                    {7, 8, 9} };

    int B[3][3] = { {2, 2, 7},
                    {6, -3, 4},
                    {3, 22, 0} };

    int C[3][3] = {0};

    int i, j, k;
```

```
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++) {
            for (k = 0; k < 3; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }

    printf("Resultant Matrix:\n");
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++) {
            printf("%d\t", C[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Output

```
muhammad-umar@umarrsthinkpad:~/Desktop/PF_Labs/Lab08_2025_EE_016$ ./task2
Resultant Matrix:
23      62      15
56      125     48
89      188     81
muhammad-umar@umarrsthinkpad:~/Desktop/PF_Labs/Lab08_2025_EE_016$
```

Task 3

Write a C program that passes a 2-D array to a function and prints the diagonal elements only.
Hint:

1. Take a square matrix, number of rows = number of columns
2. In a square matrix, the diagonal elements have row index = column index.
   Example: arr[0][0], arr[1][1], arr[2][2], ...

C Code

Open task3.c ~/Desktop/PF_Lab... ab08_2025_EE_016

```c
#include <stdio.h>

void printDiagonal(int arr[][3], int size) {
    printf("Diagonal Elements: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i][i]);
    }
    printf("\n");
}

int main() {
    int matrix[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };

    printf("Original Matrix:\n");
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%d\t", matrix[i][j]);
        }
        printf("\n");
    }
    printf("\n");

    printDiagonal(matrix, 3);

    return 0;
}
```

Output

```
muhammad-umar@umarrsthinkpad:~/Desktop/PF_Labs/Lab08_2025_EE_016$ ./task3
Original Matrix:
1       2       3
4       5       6
7       8       9

Diagonal Elements: 1 5 9
muhammad-umar@umarrsthinkpad:~/Desktop/PF_Labs/Lab08_2025_EE_016$
```

Task 4

Debug the given C code and write the corrected code with comments to highlight actual bugs in the code. Students should display the output of the corrected C code.

Bugged Code

```
#include <stdio.h>

int main() {
    int matrix[3][4] =
        {12, 7, 9, 15},
        {8, 14, 6, 10}
        {11, 5, 13, 2}
    };

    printf("Matrix with maximum of each row highlighted:\n");

    for(int i = 0; i <= 3; i++) {
        int max = 0
        for(int j = 0; j <= 4; j++) {
            if(matrix[i][j] < max)
                max += matrix[i][j];
        }

        for(int j = 0; j <= 5; j++) {
            printf("%f ", matrix[j][i]);
            if(matrix[i][j] = max)
                printf("*");
        }
        printf("\n")
    }
    return 0;
}
```

Corrected Code

task4.c
~/Desktop/PF_Labs/Lab08_2025_EE_016

```c
#include <stdio.h>

int main() {
    //Added outer braces {} and a comma after the second row
    int matrix[3][4] = {
        {12, 7, 9, 15},
        {8, 14, 6, 10},
        {11, 5, 13, 2}
    };
    printf("Matrix with maximum of each row highlighted:\n");
    //Changed condition to i < 3 (array goes 0, 1, 2)
    for(int i = 0; i < 3; i++) {
        //Added missing semicolon
        int max = 0;
        //Changed condition to j < 4
        for(int j = 0; j < 4; j++) {
            //Changed < to > to correctly find the highest number
            if(matrix[i][j] > max) {
                //Replaced += with = to properly update the max value
                max = matrix[i][j];
            }
        }
        //Changed condition to j < 4
        for(int j = 0; j < 4; j++) {
            // Changed %f to %d, and corrected indices
            printf("%d", matrix[i][j]);

            // BUG FIX 9: Changed assignment = to comparison ==
            if(matrix[i][j] == max) {
                printf("*");
            }
            printf("\t");
        }
        // missing semicolon
        printf("\n");
    }
    return 0;
}
```

Output

```
muhammad-umar@umarrsthinkpad:~/Desktop/PF_Labs/Lab08_2025_EE_016$ ./task4
Matrix with maximum of each row highlighted:
12      7       9       15*
8       14*     6       10
11      5       13*     2
muhammad-umar@umarrsthinkpad:~/Desktop/PF_Labs/Lab08_2025_EE_016$
```